Event Simulator Calibration Plan

Last updated October 21, 2026. Uses Event_Data_Stream_Simulation_v3.m, Event_Data_Diagnostics.m and your angular-rate pipeline. See the two script documentation documents for variable details.

1. Goal

Tune the simulator so its synthetic event data matches real data from your camera and telescope, within stated tolerances. Once it does, synthetic data with a known ground truth can stand in for real data when testing the angular-rate pipeline and other algorithms.

Calibration is done when two things hold: the diagnostics summary for the synthetic file matches the real file's summary (section 6), and the angular-rate pipeline behaves the same on both files.

2. What you need before starting

- **A real recording with one clear target pass** on a mostly empty field, including some time before and after the pass so the background can be measured.
- **The camera settings used for that recording:** the event threshold (bias) setting and any on-camera filters, such as hot-pixel or refractory-time filters. These change both the noise and the event rates, so they must be known and fixed.
- **The pipeline's fit for that pass.** The pipeline currently does not return the fit coefficients; add `results.px = px; results.py = py;` to `analyze_angular_rate`.
- **Ideally three or more passes** at different speeds and brightnesses. Use one for tuning and hold the others out for validation (Step 7).
- **A working run of both scripts in MATLAB.** The pixel logic was checked in a Python port, but the MATLAB files have not been run by me, so confirm that your MATLAB outputs look like the numbers in the documentation before tuning anything.

3. Which parameters you set, measure, or tune

Group	Parameters	How they are set
Fixed by hardware or format	`nx`, `ny`, `pixPitch_m`, `focalLen_m`, `polarity01`, `Delta_lam`	Take from the camera and optics. Delta_lam comes from the camera's bias setting; the paper quotes about 0.25 as the EVK3 default, so check your actual setting.
Measured directly from the real data	track values (`x0`, `y0`, `vx`, `vy`, `T_sim`), `sigma_psf`, `noiseRate` or `lambda_r_full`	Read off the diagnostics summary and the pipeline fit.
Tuned by iteration	`a`, `Phi_0`, and `lambda_r_full` in full noise mode	Steps 3 to 5.
Insensitive	`b`, `seed`, `dt` (while speed times dt stays below about 0.3 px)	Leave alone.
Held fixed by assumption	`mod_depth` = 0, and `lam_bg`	`lam_bg` trades off against `lambda_r_full`, so fix it and tune only `lambda_r_full`.
Not modeled	hot pixels, refractory period, bandwidth limits, seeing flicker	See section 8.

4. What to measure on the real data

Run Event_Data_Diagnostics.m on the real file. These are your calibration targets.

#	Metric (diagnostics line)	Simulator knob it calibrates
1	Background mean rate	`noiseRate` (clean) or `lambda_r_full` (full noise)
2	Background positive fraction	Check against the simulator's 50/50 noise
3	Pixels with at least 1 event, busiest pixel	Check the noise spread; reveals hot pixels

#	Metric (diagnostics line)	Simulator knob it calibrates
4	Events in gate per second	`Phi_0` (jointly with `a`)
5	- events: along-track median and 10th percentile	`a` (tail length)
6	+ events: along-track median and 10/90	Alignment check; should sit near 0
7	rms cross-track, + and - events	`sigma_psf`
8	Figure 2 (rate vs time): floor and plateau levels	Visual check of 1 and 4
9	Pipeline output: rate, inlier fraction, rms residual	End-to-end check

5. Procedure

Step 0. Baseline (already done once)

With default values, the pipeline recovered 1024.3 arcsec/s against a true value of about 1025.8, about 0.15% low. Repeat this on your MATLAB install to confirm the whole chain works before tuning.

Step 1. Measure the real data

1. Run the pipeline on the real pass and read out `px` and `py`.
2. Work out the time zero. The pipeline's time axis may not start at the file's first event, so check how your loader sets it and match `tZero_us` and `trackT0` in the diagnostics script.
3. Run the diagnostics on the real file. Confirm in Figure 1 that the black cross sits on the target. If it does not, the track or time zero is wrong; fix that before anything else.
4. Record every number in the log (section 9) as the Real row.

Step 2. Set the fixed and measured inputs

1. Set `nx`, `ny` and `Delta_lam` from the camera.
2. Set the track values and `T_sim` from the pipeline fit.

3. Set `sigma_psf` from a star or the target's blur: sigma is about FWHM divided by 2.355. Then check the synthetic cross-track rms against the real one in the diagnostics and adjust `sigma_psf` until they agree. The cross-track rms is not equal to sigma (it was about 2.5 px for sigma 1.5 in the example), so compare real against synthetic rather than converting directly.
4. Keep `mod_depth` at 0.

Step 3. Calibrate the background

Clean mode. Set `noiseRate` to the real background mean rate. Use the plain mean for sparse noise and the top-1%-removed mean if hot pixels or stars dominate. Then compare the positive fraction: clean mode is fixed at 50/50, so if the real value is clearly different, this mode cannot reproduce it and either a new input is needed or full noise mode should be used.

Full noise mode. Start from the sweep table in the simulator documentation and adjust `lambda_r_full` until the background mean is within 10% of the real value. Background rate is per pixel, so while tuning you can use a cropped sensor (for example 300 x 200) and a short `T_sim` to save time. The rate depends on `a` and `Delta_lam`, so tune it after those are fixed, and revisit it after Step 4.

Step 4. Calibrate the tail and the amplitude together

These two interact, so iterate.

1. **Tail (set `a`).** The tail length scales roughly with speed divided by `a`. If the simulated - along-track median is longer than the real one at the same speed, raise `a` by about the ratio, and vice versa: `a_new = a_old x (simulated tail / real tail)`. Compare the median and the 10th percentile, and compare the histogram shape in Figure 3, not only one number.
2. **Amplitude (set `Phi_0`).** Adjust `Phi_0` until the events per second in the gate match. Event count rises with brightness but less than proportionally, so use a bracketing or secant search rather than a proportional update.
3. Repeat 1 and 2 until both are within tolerance. Expect a few loops.

Step 5. Re-check the background (full noise mode only)

Changing `a` changes the background rate, so re-measure it and re-tune `lambda_r_full`, then repeat Step 4 if the tail or amplitude moved. In clean mode the noise is independent of `a`, so skip this step.

Step 6. Build the sensitivity table (once)

Vary each tuned knob by about 25% up and down, one at a time, and record how each metric moves. This replaces the expected sensitivities in section 7 with measured ones and shows which parameters are degenerate.

Step 7. Hold-out validation

With the parameters frozen, change only the track to match another real pass (different speed, direction or brightness) and compare the diagnostics against that pass without retuning. A speed change is the strongest test of `a`, because tail length in pixels should scale with speed. A brightness change tests how `Phi_0` scales. Also run the pipeline on both files and compare the number of detections, inlier fraction, rms residual and rate error.

Step 8. Freeze and record

Save the final parameter table, the seed, the metrics comparison, the camera settings, the data file names and the date.

6. Acceptance criteria

These are suggested starting tolerances, not requirements derived from your data. Adjust them to what your application needs.

Metric	Suggested tolerance
Background mean rate	within 10%
Background positive fraction	within 5 percentage points
Pixels with at least 1 event	within 5 percentage points
Gate events per second	within 15%
- along-track median	within 15% (10th percentile within 20%)
+ along-track median	within 1 px of the real value
rms cross-track (+ and -)	within 0.5 px
Pipeline inlier fraction and rms residual	within 20% of the real file

Metric	Suggested tolerance
Pipeline rate on synthetic data	recovered within the pipeline's own reported uncertainty of the truth
Hold-out passes	all of the above within 1.5 times these tolerances, with no retuning

7. Expected sensitivities (to verify in Step 6)

These are directions expected from the model, not measured results for your camera.

Knob	Background rate	Tail length	Gate events	Cross-track rms
`noiseRate` (clean)	rises directly	none	none	none
`lambda_r_full` (full noise)	falls as it approaches lam_bg	none	none	none
`a`	rises (full noise)	shortens	changes	small
`Delta_lam`	falls as it grows	shortens	falls	small
`Phi_0`	none	lengthens	rises, less than proportionally	slight increase
`sigma_psf`	none	small	changes	rises
`b`	none	none	none	none
`lam_bg`	shifts (full noise); degenerate with `lambda_r_full`	none	none	none

8. Risks and known gaps

- **Parameter degeneracy.** `a`, `Delta_lam`, `lambda_r_full` and `lam_bg` all affect the noise, so different combinations can match the same numbers. Fix the ones you can measure or read from the camera and tune as few as possible.

- **Noise structure is not modeled.** Hot pixels, correlated noise, refractory period, bandwidth limits and on-camera filters are absent, and matching the mean rate will not reproduce them. Compare the busiest pixel and the fraction of pixels that fire. If they differ a lot, the simulator needs a hot-pixel model.
- **Seeing flicker.** The paper found real central event rates around bright stars were higher than its model predicted and attributed this to atmospheric seeing. Statistics near a bright target core may not match.
- **Blur shape.** The simulator uses a Gaussian blur; a defocused or non-Gaussian PSF will differ.
- **Circuit model.** The pixel is modeled as a first-order circuit. If the real tail has a different shape, matching its median will not match its histogram, so compare shapes as well as numbers.
- **Pipeline bias in the reference.** For real data the pipeline's fit is the reference track, and its fit bias carries over. On synthetic data it read about 0.15% low, probably from edge clipping. Run the pipeline on both files and measure offsets against its fit in both cases so the bias cancels.
- **Time zero and indexing.** A wrong time zero or 0/1-based coordinate setting silently breaks the gate. Always check Figure 1 first.
- **Tuning on a single pass overfits.** Keep at least one pass for hold-out validation.
- **Unrun in MATLAB.** Confirm Step 0 on your own install.

9. Calibration log template

Run	Date	Knobs changed	Bg mean rate	Bg + fraction	Gate events/s	- median (px)	- 10th pct (px)	rms cross (px)	Pipeline rate (arcsec/s)	Notes
Real (target)										
Sim 1										
Sim 2										
Sim 3										
Hold-out real										
Hold-out sim										

10. Done when

- The final parameter set is recorded with the seed and camera settings.
- The synthetic and real diagnostics agree within the section 6 tolerances on the tuning pass and the hold-out passes.
- The pipeline gives comparable results on real and synthetic data.
- The unmodeled effects that still differ (section 8) are written down so later users know the limits.

11. Suggested first session

1. Run the simulator and diagnostics on defaults in MATLAB and confirm the outputs resemble those in the documentation.
2. Expose `px` and `py` from the pipeline.
3. Run the pipeline and diagnostics on one real pass and fill in the Real row of the log.
4. Do Steps 2 and 3 (fixed inputs and background), and stop there to review before tuning `a` and `Phi_0`.